Verification Data Flow: UI → Relayer → Blockchain

Overview

This document explains how biometric identity verification data flows through the system and what data is sent at each stage.

Architecture

```
User's Face
    ↓
UI (React App) - Processes face → Generates ZK Proof
    ↓
Relayer (Rust Server) - Parses proof → Sends transaction
    ↓
Blockchain (Ethereum) - Verifies proof → Logs event
```

Stage 1: User's Face → Biometric Data

Input: Face Scan

```
MediaPipe FaceMesh detects 468 3D landmarks
Example points: nose, eyes, mouth, chin positions
Format: {x: 0.432, y: 0.678, z: 0.123} × 468 points
```

Processing: Extract Stable Ratios

```
// Extract 4 geometric ratios (invariant to distance, angle, lighting)
ratios = [
  distance(nose, leftEye) / distance(leftEye, rightEye),   // 0.7234
  distance(nose, rightEye) / distance(leftEye, rightEye),  // 0.7156
  distance(mouth_left, mouth_right) / distance(eyes),       // 1.2341
  distance(nose, chin) / distance(eyes)                     // 1.8923
]
```

Quantization: Convert to Integers

```
// Multiply by 100,000 to get integers (needed for cryptography)
quantized = [72340, 71560, 123410, 189230]
```

Double Hashing: Create Identity

```
// Level 1: Create Secret ID
secretID = Poseidon([72340, 71560, 123410, 189230])
// Result: "14235892..." (your private biometric secret)
```

```
// Level 2: Create Public Commitment
commitment = Poseidon([secretID])
// Result: "25987123..." (your public identity)
```

Privacy Note:

- 🔒 Face data never leaves the browser
- 🔒 Only hashes are stored/transmitted
- 🔒 Cannot reverse engineer face from hash

Stage 2: UI → Generates Zero-Knowledge Proof

Proof Generation (SnarkJS)

```
// User inputs (private)
inputs = {
  secretId: "14235892...",          // 🔒 Private (your biometric hash)
  identityCommitment: "25987123...", // ✅ Public
  app_address: "0x00000...",         // ✅ Public (which app)
  user_wallet: "0x16Dd...",          // ✅ Public (your wallet)
  nullifier: "52054355..."           // ✅ Public (unique per app+user)
}

// Generate proof using circuit
proof = snarkjs.groth16.fullProve(inputs, circuit.wasm, proving.key)
```

Proof Output Format (Groth16)


```
{
  "proof": {
    "pi_a": [

"7896933342017620227527520551720444641191159076469243331451934108320838396128",
      "3029012762734493991814172554840841996452988470082317470022676337927153197 36",
      "1"
    ],
    "pi_b": [

["21574675583010863025866989275808290247882323371859260152299823491306093118562",

"18563609877636348656780559883626961127083498001598341646284837547225273193096"],

["6639054353456365131962810212352875162254921470385187293202964367861961213951",

"8235293273310111642519128281393214442121401117589992775500413337313259878058"],
      ["1", "0"]
    ],
    "pi_c": [
```

```
"16993018567310881185875685833561738767541549073588659603007359432417132169896",

"30534717770009833537821449762833208097914842341716849779594242953797935102 11",
      "1"
    ]
  },
  "publicSignals": [
    "16830023662952245248302784979873025312420635066854034736718457824433595915919",
// commitment
    "12296296352543114541258143884573379005655133212",
// app_address
    "13052892469888433921711205970789183637570105157 4",
// user_wallet
    "52054355703520493903756695749848894718976017718101730338027334769554785586 59"
// nullifier
  ]
}
```

What is This Proof?

pi_a, pi_b, pi_c = Cryptographic proof components

Think of them as **three sealed envelopes** that together prove:

- ✅ You know the secret (biometric data)
- ✅ The commitment matches your identity
- ✅ The nullifier is correctly calculated
- ❌ WITHOUT revealing your secret!

Technical Details:

- `pi_a` : 2 coordinates on elliptic curve (BN254)
- `pi_b` : 4 coordinates (2x2 matrix, special G2 group for pairing)
- `pi_c` : 2 coordinates on elliptic curve
- Each coordinate: 254-bit number (~77 decimal digits)

Nullifier Purpose:

- Unique per (user + app) combination
- Prevents replay attacks (can't reuse same proof)
- Calculated: `Poseidon(secretID, app_address, wallet_address)`

Stage 3: UI → Relayer (HTTP API)

HTTP Request

```
POST http://localhost:3001/relay
Content-Type: application/json

{
  "proof": { "pi_a": [...], "pi_b": [...], "pi_c": [...] },
```

```
  "publicSignals": [...]
}
```

Data Sent Over Network

- **Size**: ~2-3 KB JSON
- **Transport**: HTTP POST (unencrypted in local testing, use HTTPS in production)
- **Contents**: Proof + public signals (no secrets!)

Stage 4: Relayer → Parses & Transforms Data

Circom Format → Solidity Format

The relayer converts from Circom's output format to what Ethereum expects:

```
// Remove padding elements (the "1"s and "0"s)
a = [pi_a[0], pi_a[1]]                      // Take first 2 elements
b = [[pi_b[0][0], pi_b[0][1]],             // Take first 2 sub-arrays
     [pi_b[1][0], pi_b[1][1]]]
c = [pi_c[0], pi_c[1]]                     // Take first 2 elements
nullifier = publicSignals[3]                // Extract from index 3

// Solidity format ready!
```

Transformation Example

```
Before (Circom):
pi_a = ["123...", "456...", "1"]        // 3 elements

After (Solidity):
a = ["123...", "456..."]                // 2 elements (removed padding)
```

Stage 5: Relayer → Blockchain (Transaction)

Smart Contract Function Called

```
function verify(
    uint256[2] calldata a,        // First proof component
    uint256[2][2] calldata b,     // Second proof component
    uint256[2] calldata c,        // Third proof component
    uint256[1] calldata input     // Nullifier (public input)
) external returns (bool)
```

Transaction Data Sent

Component a: uint256[2]

```
Type: 2 large numbers (elliptic curve coordinates)
Size: 64 bytes (32 bytes × 2)
Purpose: First piece of cryptographic evidence

Example:
a[0] = 7896933342017620227527520551720444641191159076469243331451934108320838396128
a[1] = 3029012762734493991814172554840841996452988470082317470022676337927153197 36

What it represents:
- Point on BN254 elliptic curve
- X and Y coordinates
- Encodes proof of knowledge in encrypted form
```

Component b: uint256[2][2]

```
Type: 4 large numbers (2×2 matrix of curve coordinates)
Size: 128 bytes (32 bytes × 4)
Purpose: Second piece of evidence (pairing-based cryptography)

Example:
b[0][0] =
21574675583010863025866989275808290247882323371859260152299823491306093118562
b[0][1] =
18563609877636348656780559883626961127083498001598341646284837547225273193096
b[1][0] = 6639054353456365131962810212352875162254921470385187293202964367861961213951
b[1][1] = 8235293273310111642519128281393214442121401117589992775500413337313259878058

What it represents:
- Point on twisted BN254 curve (G2 group)
- Requires 2 field elements per coordinate = 4 numbers total
- Used in pairing check during verification
```

Component c: uint256[2]

```
Type: 2 large numbers (elliptic curve coordinates)
Size: 64 bytes (32 bytes × 2)
Purpose: Third piece of cryptographic evidence

Example:
c[0] = 16993018567310881185875685833561738767541549073588659603007359432417132169896
c[1] = 3053471777000983353782144976283320809791484234171684977959424295379793510211

What it represents:
- Another point on BN254 curve
- Completes the proof structure
- Together with a and b, forms complete verification equation
```

Input (Nullifier): uint256[1]

```
Type: 1 large number (hash output)
Size: 32 bytes
```

```
Purpose: Unique identifier for this verification

Example:
input[0] =
5205435570352049390375669574984889471897601771810173033802733476955478558659

What it represents:
- NOT part of the proof itself (it's public input)
- Hash of (secretID, app_address, wallet_address)
- Prevents replay attacks
- Different for each app you verify with
```

Total Data Size on Blockchain

```
a: 2 uint256 = 64 bytes
b: 4 uint256 = 128 bytes
c: 2 uint256 = 64 bytes
nullifier: 1 uint256 = 32 bytes
────────────────────────
Total: 9 uint256 = 288 bytes
```

Stage 6: Blockchain Verification & Response

How Proof Verification Works (Elliptic Curve Mechanism)

The blockchain verifies the proof using **elliptic curve pairing** - a mathematical operation that checks if the proof is valid.

The Verification Equation

```
e(a, b) = e(c, public_inputs)

Where:
- e() = pairing function (multiplies points on elliptic curves)
- a, c = points on G1 curve (BN254)
- b = point on G2 curve (BN254 twisted)
- public_inputs = nullifier encoded as curve point
```

Step-by-Step Verification

Step 1: Load Proof Components

```
Contract receives:
- a: [uint256, uint256] → Convert to G1 point
- b: [[uint256, uint256], [uint256, uint256]] → Convert to G2 point
- c: [uint256, uint256] → Convert to G1 point
- nullifier: [uint256] → Public input
```

Step 2: Elliptic Curve Pairing Check

```
function verifyProof(
    uint256[2] a,
    uint256[2][2] b,
    uint256[2] c,
    uint256[1] input
) returns (bool) {
    // Perform pairing check
    // Checks: e(a, b) == e(c, [public_inputs + verification_key])

    return pairingCheck(a, b, c, input);
}
```

What the Pairing Check Does:

```
1. Maps proof points to elliptic curve groups
2. Performs two pairing operations:
   - Pair(a, b) → Result1
   - Pair(c, verification_key + public_input) → Result2
3. Check if Result1 == Result2
4. Returns true if equal, false otherwise
```

Why This Works:

- The proof (a, b, c) can ONLY satisfy the equation if:
  - ✅ You know the secret (biometric data)
  - ✅ The circuit computations were correct
  - ✅ The public inputs match
- If any value is wrong or fake, the equation won't balance
- Mathematics guarantees this is impossible to fake

Data Size Breakdown

```
a: 2 uint256 = 64 bytes   (G1 point: x, y coordinates)
b: 4 uint256 = 128 bytes  (G2 point: 2 coordinates × 2 field elements)
c: 2 uint256 = 64 bytes   (G1 point: x, y coordinates)
nullifier: 1 uint256 = 32 bytes (public input hash)
────────────────────────
Total: 9 uint256 = 288 bytes
```

Why Different Sizes:

- G1 points (a, c): Regular elliptic curve points = 2 coordinates
- G2 point (b): Twisted curve requiring extension field = 4 values
- This structure enables efficient pairing-based verification

---

Stage 7: Response Back to UI

Blockchain → Relayer

```
// Relayer receives transaction receipt
let receipt = pending_tx.await?;

// Extract result
let tx_hash = receipt.transaction_hash;
let status = receipt.status; // 1 = success, 0 = failed
let block_number = receipt.block_number;
```

Relayer → UI (HTTP Response)

```
{
  "success": true,
  "message": "Proof verified successfully",
  "tx_hash": "0xa8b79f0b6132fa08b325648e2ccb3a6c5d33e9a83f2aaa8e60cd85de4e796b22"
}
```

UI Display

```
// UI receives response
const result = await fetch('http://localhost:3001/relay', {...});
const data = await result.json();

if (data.success) {
  // Show success popup modal
  setShowModal(true);
  setTxHash(data.tx_hash);
  // Display: "✅ Verification Complete!"
} else {
  // Show error message
  alert(`Verification Failed: ${data.message}`);
}
```

Complete Round Trip

```
UI: "Here's my proof"
  ↓
Relayer: "Sending to blockchain..."
  ↓
Blockchain: "Verifying... ✅ Valid!"
  ↓
Relayer: "Success! Here's tx hash"
  ↓
UI: "🎉 Verification Complete!"
```

Complete Data Flow Summary

```
┌──────────────────┐
│  User's Face     │
│  468 landmarks   │
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Extract 4       │
│  ratios          │ 🔒 All processing
│  [0.72, 0.71,    │    happens in
│   1.23, 1.89]    │    browser
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Quantize        │
│  [72340,71560,   │
│   123410,189230]│
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Hash Level 1    │
│  secretID =      │
│  Poseidon(...)   │
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Hash Level 2    │
│  commitment =    │
│  Poseidon(...)   │
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Generate Proof │
│  [a, b, c]       │
│  + nullifier     │
└──────────────────┘
         │
         ↓  HTTP POST
┌──────────────────┐
│  Relayer         │
│  Parse & Format │
└──────────────────┘
         │
         ↓  Ethereum Transaction
┌──────────────────┐
│  Blockchain      │
│  Elliptic Curve │ ✅ Pairing check
```

```
│  Verification   │    verifies proof
└─────────────────┘
         │
         │
         ↓  Transaction Receipt

┌─────────────────┐
│  Relayer        │
│  Extract Result │
│  & TX Hash      │
└─────────────────┘
         │
         │
         ↓  HTTP Response

┌─────────────────┐
│  UI             │
│  Show Success   │ 🎉 Display
│  Modal Popup    │    confirmation
└─────────────────┘
```

Key Takeaways

1. **Privacy First**: Face data never leaves the browser, only cryptographic hashes are sent
2. **Zero-Knowledge**: Proof demonstrates knowledge without revealing the secret
3. **Compact Data**: Only 288 bytes sent to blockchain (9 large numbers)
4. **Elliptic Curve Verification**: Uses pairing check to verify proof mathematically
5. **Complete Round Trip**: UI → Relayer → Blockchain → Relayer → UI with success confirmation

---

Document Version: 1.1
Last Updated: February 15, 2026
Status: Verification mechanism explained